

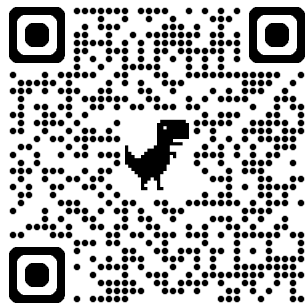
Claude Codeの進化と 各機能の活かし方

Speaker: Oikon

(Mar. 6, 2026)
企画: Findyさま



自己紹介



Oikon (@oikon48)

- Software Engineer
- 趣味: 個人開発, AIツール研究
- Claude Code歴: 2025年3月～



前回登壇: Claude Codeはじめてガイド

想定するオーディエンスと今日のゴール

想定するオーディエンス

- Claude Codeの各種機能（Skills, Hooks, Sub-Agents等）の概要を知りたい
- 各機能の使いどころを理解し、実際に使用するイメージを掴みたい

今日のゴール

各機能について「概要・使いどころ」を解説
Claude Codeの機能活用イメージを持ち帰ってもらう

今日、話すこと

Part 1: 全体像

- Best Practices for Claude Code
- 機能の全体マップ

Part 2: 各機能について

- CLAUDE.md
- Permissions
- Hooks
- Skills
- Subagents
- MCP
- Plugins
- Agent Teams

Part 3: まとめ

- 個人的に意識していること / Q&A

Part 1: 全体像



Best Practices for Claude Code

Getting started

- Overview
- Quickstart
- Changelog ↗

Core concepts

- How Claude Code works
- Extend Claude Code
- Store instructions and memories
- Common workflows

Best practices

Platforms and integrations

- Remote Control
- Claude Code on the web
- Claude Code on desktop >

CORE CONCEPTS

Best Practices for Claude Code

📄 Copy page ▾

Tips and patterns for getting the most out of Claude Code, from configuring your environment to scaling across parallel sessions.

Claude Code is an agentic coding environment. Unlike a chatbot that answers questions and waits, Claude Code can read your files, run commands, make changes, and autonomously work through problems while you watch, redirect, or step away entirely.

This changes how you work. Instead of writing code yourself and asking Claude to review it, you describe what you want and Claude figures out how to build it. Claude explores, plans, and implements.

But this autonomy still comes with a learning curve. Claude works within certain constraints you need to understand.

This guide covers patterns that have proven effective across Anthropic's internal teams and for engineers using Claude Code across various codebases, languages, and environments. For how the agentic loop works under the hood, see [How Claude Code works](#).

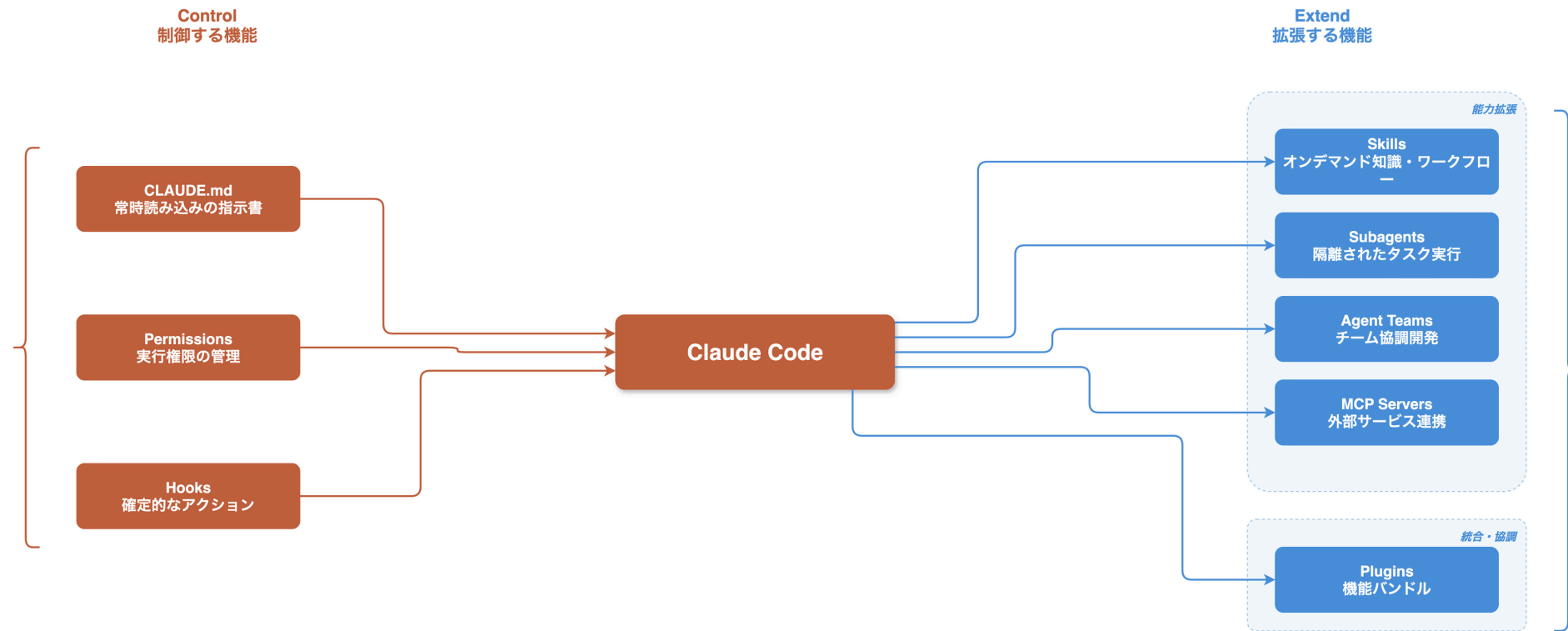
参考: code.claude.com/docs/en/best-practices

☰ On this page

- Give Claude a way to verify its work
 - Explore first, then plan, then code
 - Provide specific context in your prompts
 - Provide rich content
- Configure your environment
 - Write an effective CLAUDE.md
 - Configure permissions
 - Use CLI tools
 - Connect MCP servers
 - Set up hooks
 - Create skills
 - Create custom subagents
 - Install plugins
- Communicate effectively
 - Ask codebase questions
 - Let Claude interview you

機能全体のイメージ

Claude Codeの各機能がどのように連携し、Claudeを制御・拡張するか



Part 2: 各機能の解説

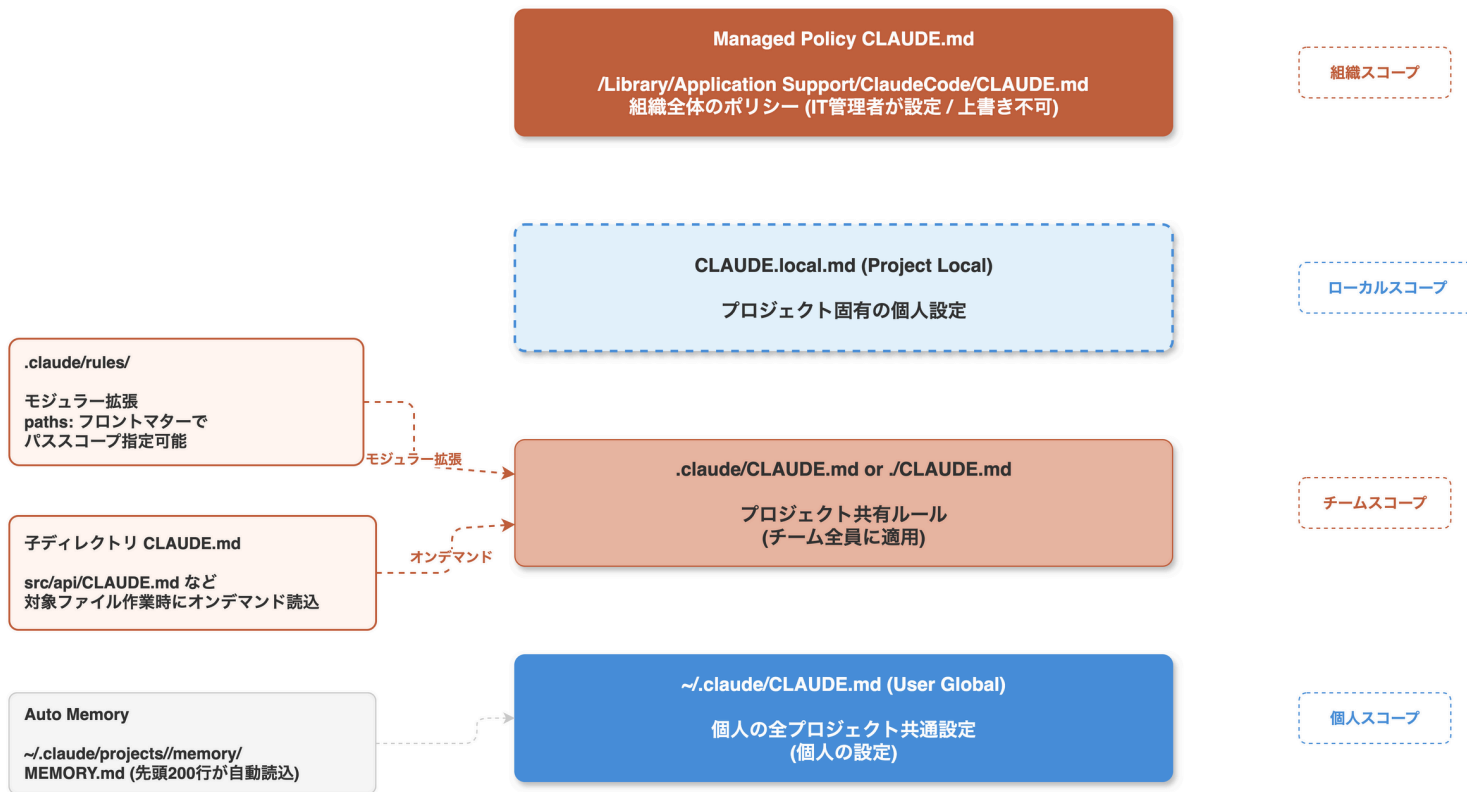


CLAUDE.md

CLAUDE.md 階層構造

優先度 高

優先度 低

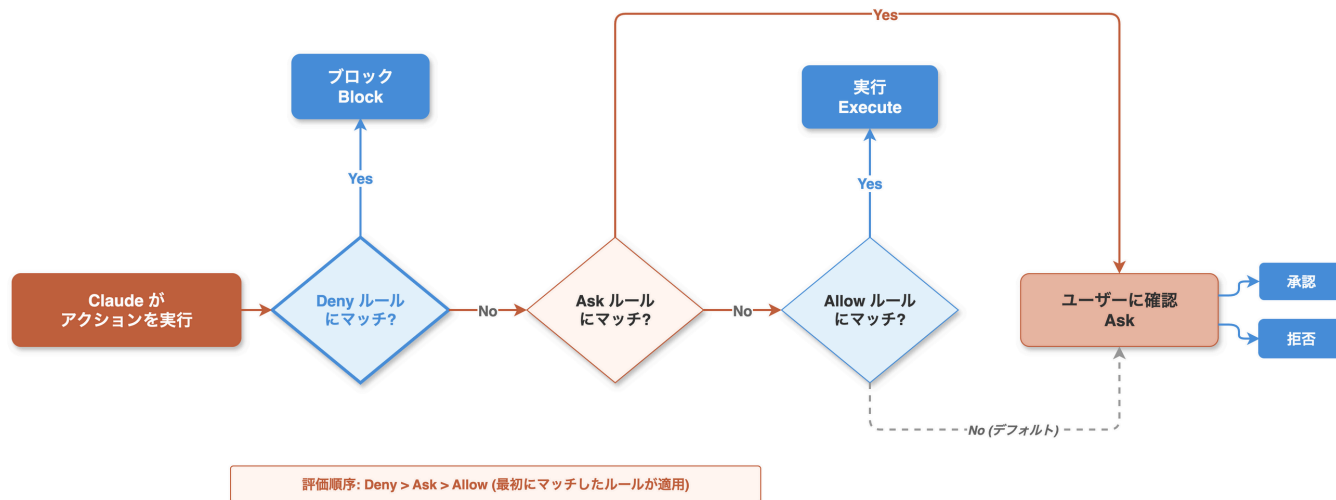


参考: code.claude.com/docs/en/memory

Permissions

Claude Codeの操作を許可/拒否で制御する仕組み(setting.jsonで設定)

Permissions フロー



Permission モード (5種類)

default	標準の確認モード
acceptEdits	ファイル編集を自動承認
plan	読み取り専用の分析
dontAsk	未承認は自動拒否
bypassPermissions	全チェック省略 (CI/CD向け)

`--dangerously-skip-permissions` △ 全確認チェックをスキップ

`/sandbox` - OS レベルの隔離実行環境 (FS・ネットワーク制限)

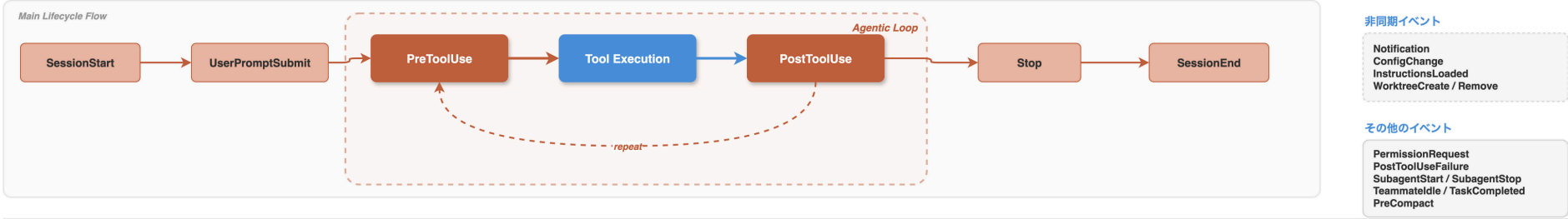
ルール構文例

```
Bash(npm run *) Read(/path/to/file)
Edit(*). Write(*)
WebFetch(domain:example.com)
mcp_servername_toolname
Agent(SecurityReviewer)
```

Hooks

Claude Codeの動作を制御する機能

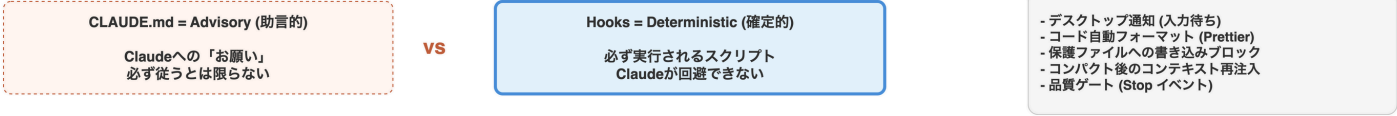
Hooks ライフサイクル



Hook タイプ (4種類)



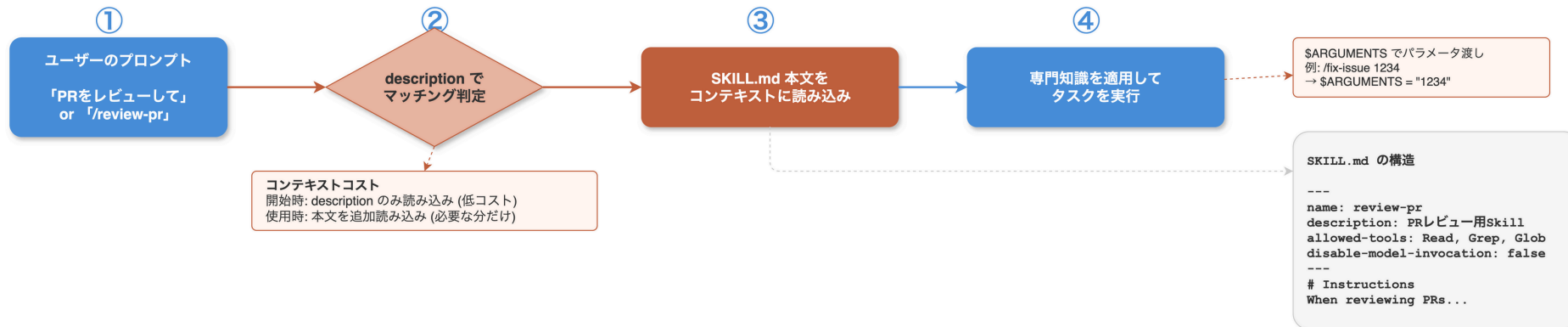
CLAUDE.md vs Hooks



Skills (Slash Commands)

オンデマンドでロードされる専門知識モジュール

Skills: オンデマンド読み込みとコンテキスト最適化



CLAUDE.md vs Skills 比較

CLAUDE.md — 常時読み込み (Always Loaded)
毎回コンテキストを消費

Skills — オンデマンド読み込み (On-Demand)
必要なときだけ読み込み

呼び出し方法

自動検出
Claude が description から自動判断

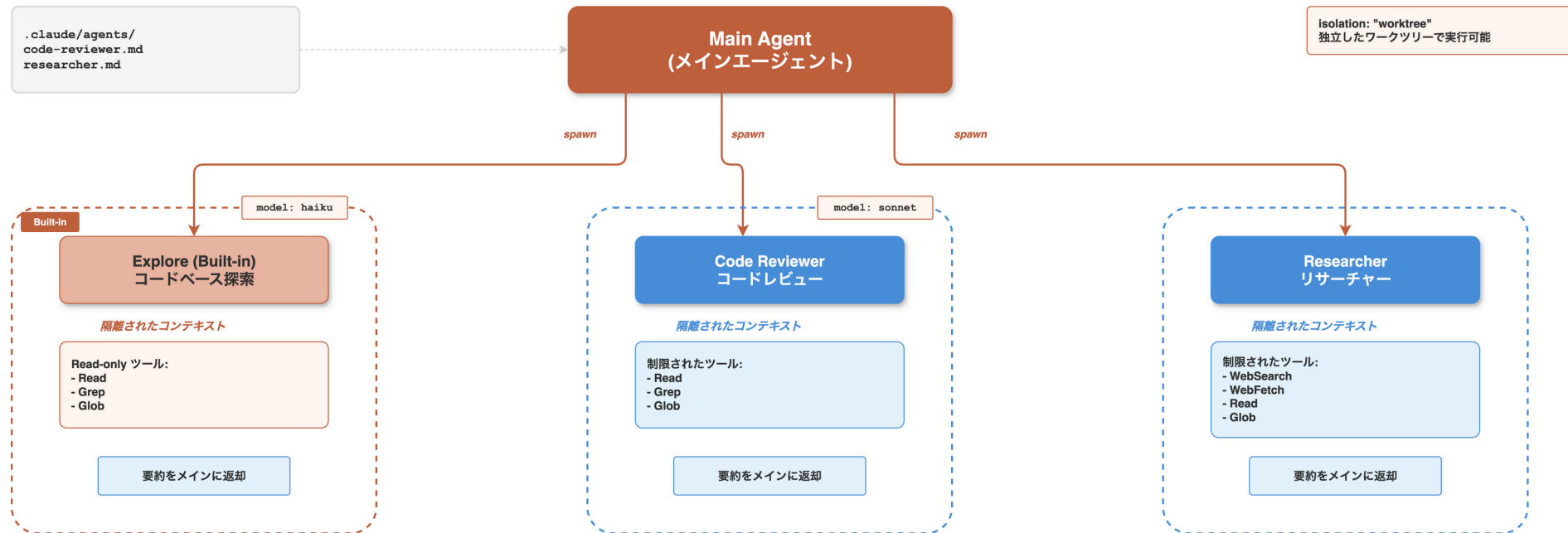
手動呼び出し
/skill-name コマンドで直接起動

disable-model-invocation: true
自動検出を無効化し手動起動のみに制限
deploy 等の副作用があるSkillに推奨

Subagents

独立したコンテキストで専門タスクを実行するエージェント

Custom Subagents アーキテクチャ

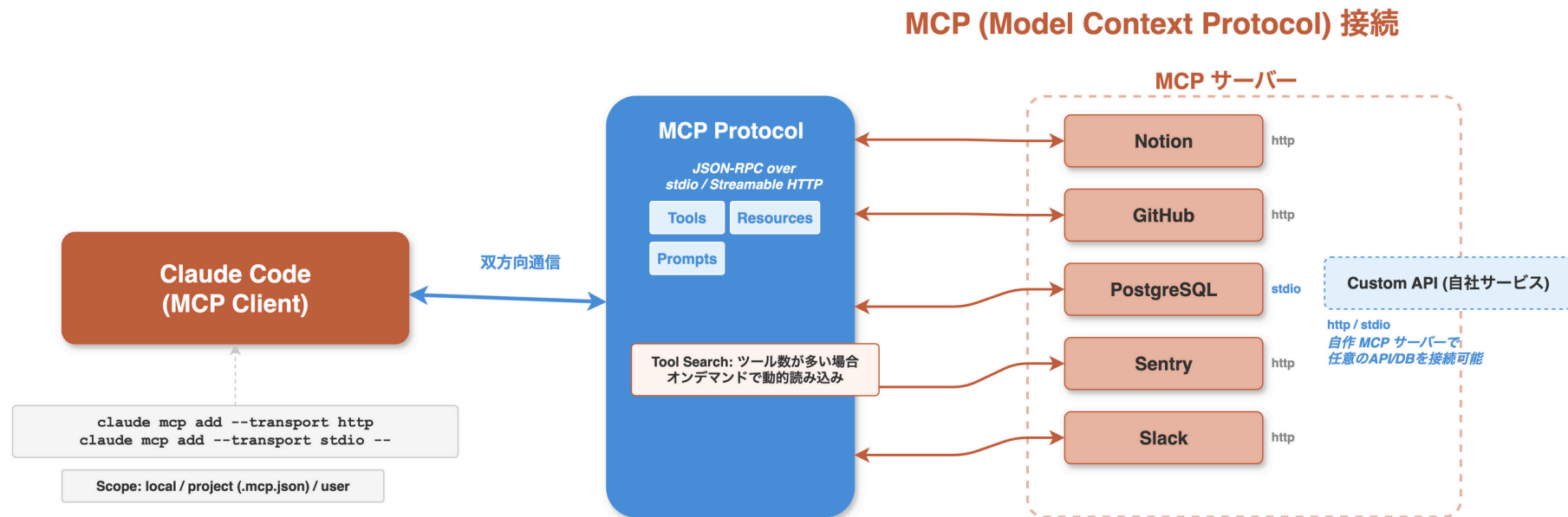


Key: 各Subagentは独立したコンテキストとツール制限を持つ

参考: code.claude.com/docs/en/sub-agents

MCPサーバー連携

外部サービスと標準プロトコルで接続



CLIツール連携

CLI tools are the **most context-efficient way** to interact with external services.

CLIツールの例

- gh (GitHub CLI): PR作成、Issue管理
- aws / gcloud: クラウドリソース操作
- sentry-cli: エラートラッキング

新しいCLIツールを学ばせる

```
> Use 'foo-cli --help' to learn about foo tool
```

Plugins

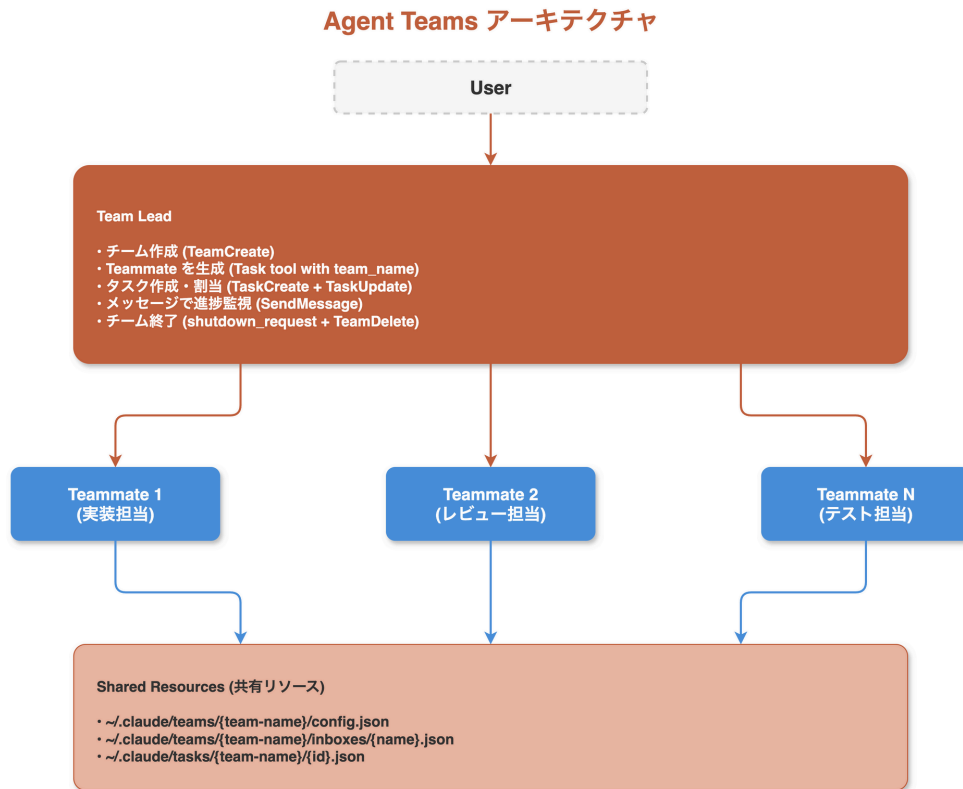
チームに配布可能なバージョン管理できる拡張パッケージ

Plugins = 機能バンドル



Agent Teams

複数のClaude Codeインスタンスが**共有タスクリスト**と**メッセージング**で協調する
※experimental



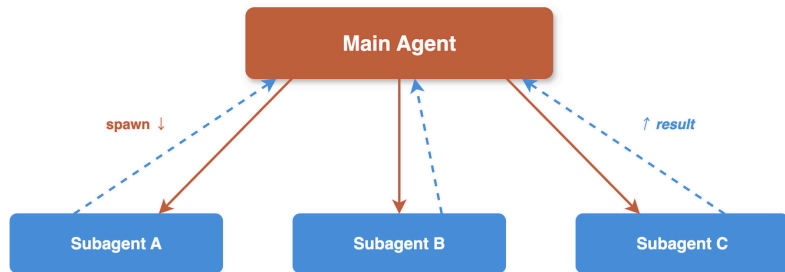
参考: code.claude.com/docs/en/agent-teams / oikon48.dev (Agent Teams解説)

Subagents vs Agent Teams

Subagents vs Agent Teams

Subagents

1対多の委任モデル

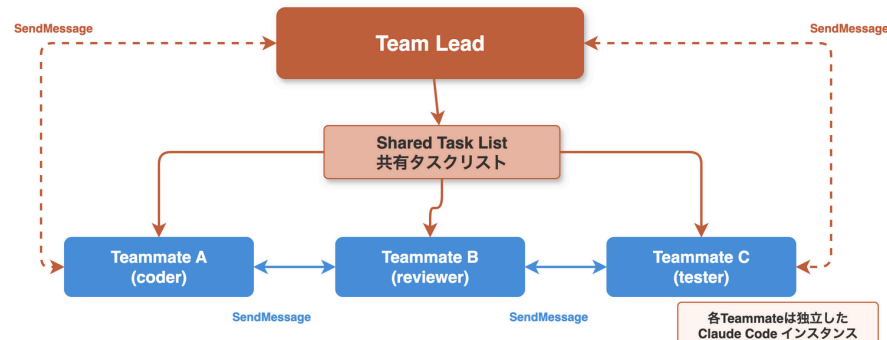


特徴:

- 各Subagentは独立して動作
- Subagent同士の通信なし
- コンテキストは隔離
- Main Agentが統合
- 1つのセッション内で完結

Agent Teams

チーム協調モデル (experimental)



特徴:

- Team Leadがタスク割り当て
- Teammate同士が通信可能
- 共有タスクリストで進捗管理
- 複数セッションが並行動作
- 大規模タスクの分担に最適

比較まとめ

Subagents

通信
セッション
ユースケース

なし (隔離)
単一セッション内
単発タスク委任・結果返却

Agent Teams

SendMessage (双方向)
複数セッション並行
大規模機能開発・チーム分担

Part 3: まとめ + Q&A



Claude Codeを使う時に個人的に意識していること

#1 参照可能なコンテキストを用意

- プロジェクトドキュメント、外部ライブラリ、Specドキュメント

#2 ツールを整備する

- ツールはClaudeの手足。使い方も必要に応じて提供する

#3 Claudeに検証手段を与える

- テスト、スクリーンショット、期待出力を提供
- Claude自身でフィードバックサイクルを回せるようにする

#4 細部よりも全体把握する

- Claudeに何をしているか聞く（別セッション, /fork, /rewind）
- ASCIIなど図を書かせる

宣伝

Claude Code Meetup Tokyo #3 (3/12, Thr)

AI 駆動開発勉強会 + AIAU共催イベント

Claude Code Meetup Japan #3

自律型エージェントが切り拓く、AI駆動開発の「新境地」
「Claude Code」の最新機能と実践テクニック！！

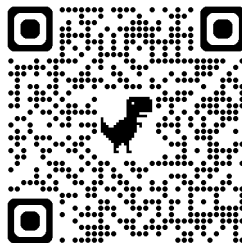
2026 **03.12** Thursday

18:40 - 21:00 東京ガーデンテラス紀尾井町 LINEヤフー株式会社



まとめ

- Claude Codeは**制御する機能**と**拡張する機能**がある
- 各機能が何ができるのかイメージして、これできるかな？と試すのが大事



@oikon48

Xでいつでも質問や相談してください

Q&A

ご質問をお待ちしています